

MYSQL TRIGGERS

Practical Examples and Student Exercises using MySQL Workbench

What is a Trigger in MySQL?

A Trigger is a database object that automatically executes when a specified event occurs on a table.

Note: You don't call a trigger manually. MySQL automatically executes it when INSERT, UPDATE, or DELETE happens.

Why do we use Triggers?

Triggers are mainly used for:

- **Auditing** – record who/what was changed
- **Validation** – prevent invalid data
- **Automatic calculations**
- **Maintaining related/summary data**
- **Logging INSERT, UPDATE and DELETE operations**
- **Enforcing business rules**

Types of MySQL Triggers

MySQL triggers are classified based on **when** they execute and **what event** activates them.

Timing

1. BEFORE
2. AFTER

Events

1. INSERT
2. UPDATE
3. DELETE

● First Trigger Example – AFTER INSERT

Create an employees table:

```
213 -- SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
214
215 ● CREATE TABLE employee(
216     emp_id INT PRIMARY KEY,
217     emp_name VARCHAR(50),
218     dept_id INT,
219     salary DECIMAL(10,2)
220 );
221 ● CREATE TABLE employee_audit(
```

Create an audit table:

```
221 ● CREATE TABLE employee_audit(
222     audit_id INT AUTO_INCREMENT PRIMARY KEY,
223     emp_id INT,
224     emp_name VARCHAR(50),
225     action_type VARCHAR(20),
226     action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
227 );
```

Now create the trigger:

```
227 DELIMITER //
228 ● CREATE TRIGGER after_employee_insert
229 AFTER INSERT ON employee
230 FOR EACH ROW
231 BEGIN
232     INSERT INTO employee_audit
233     (emp_id, emp_name, action_type)
234     VALUES
235     (NEW.emp_id, NEW.emp_name, 'INSERT');
236 END //
```

Now insert an employee:

```

25      (NEW.emp_id, NEW.emp_name, INSERT );
26  END //
27  DELIMITER ;
28  • INSERT INTO employee
29  VALUES (106, 'Arjun', 1, 52000);
30
31

```

➤ You **didn't call the trigger**. It automatically executes.

```

240  VALUES (106, 'Arjun', 1, 52000);
241  • SELECT * FROM employee_audit;
242

```

	audit_id	emp_id	emp_name	action_type	action_time
▶	1	106	Arjun	INSERT	2026-08-10 11:23:52
*	NULL	NULL	NULL	NULL	NULL

● Second Trigger Example – AFTER DELETE

Suppose Employee deleted:

```

242
243  • DELETE FROM employee
244  WHERE emp_id = 106;
245
246

```

#	Time	Action	Message
73	11:23:27	CREATE TABLE employee(emp_id INT PRIMARY KEY, emp_name VARCHA...	0 row(s) affected
74	11:23:36	CREATE TABLE employee_audit(audit_id INT AUTO_INCREMENT PRIMARY ...	0 row(s) affected
75	11:23:42	CREATE TRIGGER after_employee_insert AFTER INSERT ON employee FOR EA...	0 row(s) affected
76	11:23:52	INSERT INTO employee VALUES (106, 'Arjun', 1, 52000)	1 row(s) affected
77	11:24:20	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned
78	11:25:17	DELETE FROM employee WHERE emp_id = 106	1 row(s) affected

We can automatically record that deletion.

```
• CREATE TRIGGER after_employee_delete
  AFTER DELETE ON employee
  FOR EACH ROW
  BEGIN
    INSERT INTO employee_audit
    (emp_id, emp_name, action_type)
    VALUES
    (OLD.emp_id, OLD.emp_name, 'DELETE');
  END //
  DELIMITER ;
```

alt Grid | Filter Rows: | Edit:

emp_id	emp_name	dept_id	salary
NULL	NULL	NULL	NULL

● **Third Trigger Example--AFTER UPDATE Trigger:**

We can record the update.

```
266 • UPDATE employee
267   SET salary = 60000
268   WHERE emp_id = 101;
269
270 • drop trigger if exists after_employee_update;
271   DELIMITER //
272 • CREATE TRIGGER after_employee_update
273   AFTER UPDATE ON employee
274   FOR EACH ROW
275   BEGIN
```

```

274 FOR EACH ROW
275 BEGIN
276     INSERT INTO employee_audit
277     (emp_id, emp_name, action_type)
278     VALUES
279     (NEW.emp_id, NEW.emp_name, 'UPDATE');
280 END //
281 DELIMITER ;
282 • select * from employee;

```

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	John	1	60000.00
*	NULL	NULL	NULL	NULL

● Fourth Trigger Example--BEFORE INSERT Trigger:

Employee salary cannot be less than ₹20,000.

```

283
284 DELIMITER //
285 • CREATE TRIGGER check_salary
286 BEFORE INSERT ON employee
287 FOR EACH ROW
288 BEGIN
289     IF NEW.salary < 20000 THEN
290         SIGNAL SQLSTATE '45000'
291         SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
292     END IF;
293 END //
294 DELIMITER ;
295 • INSERT INTO employee VALUES (107, 'Ravi', 1, 15000);
296 • INSERT INTO employees VALUES (108, 'Meena', 1, 30000);

```

Now, run this Query

INSERT INTO employees

VALUES (107, 'Ravi', 1, 15000);

ports
ema Setup

```

310
311 • UPDATE employee
312 SET salary = 40000
313 WHERE emp_id = 101;
314
315
316

```

mas

Output

Action Output

#	Time	Action	Message
95	11:41:03	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employees F...	0 row(s) affected
96	11:41:12	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	Error Code: 1359. Trigger already exists
97	11:41:42	drop trigger if exists prevent_salary_reduction	0 row(s) affected
98	11:41:46	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	0 row(s) affected
99	11:42:04	UPDATE employees SET salary = 40000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
100	11:42:46	UPDATE employee SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed

Context Help

If the old salary was ₹50,000, the trigger rejects the update.

But, if you run the below update query, it succeeds as you are not reducing salary

up

```

314
315 • UPDATE employees
316 SET salary = 60000
317 WHERE emp_id = 101;
318
319

```

Output

Action Output

#	Time	Action	Message
96	11:41:12	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	Error Code: 1359. Trigger already exists
97	11:41:42	drop trigger if exists prevent_salary_reduction	0 row(s) affected
98	11:41:46	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	0 row(s) affected
99	11:42:04	UPDATE employees SET salary = 40000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
100	11:42:46	UPDATE employee SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed
101	11:44:23	UPDATE employees SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```

315 • UPDATE employee
316 SET salary = 70000
317 WHERE emp_id = 101;
318 • select * from employee;

```

Result Grid

emp_id	emp_name	dept_id	salary
101	John	1	70000.00
108	Meena	1	30000.00
NULL	NULL	NULL	NULL

Here, after salary update

OLD.salary $\rightarrow$ 60000

NEW.salary $\rightarrow$ 70000